

Singleton Pattern Implementations

A Data-Driven Analysis of MongoDB Connection Managers

Abstract

This report presents an empirical evaluation of six distinct Python implementations of the Singleton design pattern. The objective is to analyze the performance and thread-safety of each method when used to manage a MongoDB database connection. The findings presented herein are strictly derived from observed experimental data, encompassing instance validation, concurrent multithreading execution, and runtime performance benchmarks.

1 Instance Validation Results

The first experiment validates whether each implementation successfully restricts instantiation to a single object in a single-threaded environment. Each class was instantiated three consecutive times, and the memory address (ID) of the returned object was recorded.

Implementation	Call 1 ID	Call 2 ID	Call 3 ID	Result
Eager	2451652573392	2451652573392	2451652573392	Single instance
Lazy	2451652015952	2451652015952	2451652015952	Single instance
Synchronized	2451724907472	2451724907472	2451724907472	Single instance
Double-Checked	2451725015120	2451725015120	2451725015120	Single instance
Bill Pugh	2451725074512	2451725074512	2451725074512	Single instance
Enum	2451724578896	2451724578896	2451724578896	Single instance

Observation: All six implementations correctly returned the identical memory ID across multiple calls. This confirms that the foundational Singleton logic functions correctly for all methods when executed sequentially.

2 Multithreading Results

To evaluate thread safety, ten concurrent threads were spawned, each attempting to instantiate the connection manager simultaneously. A thread-safe Singleton must yield exactly 1 unique ID across all threads.

Implementation	Unique IDs Detected	Outcome
Eager	1	THREAD-SAFE
Lazy	1	THREAD-SAFE
Synchronized	1	THREAD-SAFE
Double-Checked	1	THREAD-SAFE
Bill Pugh	1	THREAD-SAFE
Enum	1	THREAD-SAFE

Observation: Based strictly on the empirical terminal output, all implementations, including the basic Lazy initialization, returned exactly 1 unique ID and survived the concurrent execution test. While Lazy initialization is theoretically vulnerable to race conditions (which the 50 ms sleep attempts to expose), the specific execution environment and threading conditions of this test run resulted in safe execution across the board. Pre-instantiated methods (Eager, Enum) and locked methods (Synchronized, Double-Checked, Bill Pugh) behaved safely as architecturally intended.

3 Performance Measurements

Performance was measured across two metrics: the time taken to initialize the very first instance, and the average time taken to access the instance over 50,000 repeated calls.

3.1 Initialization Time

Implementation	Initialization Time (ms)
Eager	0.005 ms
Enum	0.003 ms
Double-Checked	178.265 ms
Bill Pugh	213.841 ms
Synchronized	397.543 ms
Lazy	560.294 ms

Observation: The Eager and Enum methods exhibit near-zero initialization times during the timed phase (0.005 ms and 0.003 ms, respectively) because their `MongoClient` objects are constructed at module load time rather than inside the timed call. The other methods exhibit significantly higher times (178 ms to 560 ms) due to the intentional 50 ms simulation sleep and runtime instantiation overhead.

3.2 Repeated-Access Time (50,000 iterations)

Implementation	Avg Access Time (μ s)
Eager	0.1082 μ s
Double-Checked	0.1110 μ s
Bill Pugh	0.1240 μ s
Lazy	0.1242 μ s
Enum	0.1289 μ s
Synchronized	0.3349 μ s

Observation: Access time highlights the critical flaw of the Synchronized method. At 0.3349 μ s per call, it is nearly three times slower than the other implementations because it must acquire and release a thread lock on every single access. Conversely, Double-Checked Locking (0.1110 μ s) effectively bypasses the lock after the first creation, performing on par with lock-free methods like Eager (0.1082 μ s).

4 Comparative Summary

Conclusion based on observed data:

- **Fastest Implementations:** Eager and Enum are the fastest overall, boasting near-instant runtime initialization and highly efficient access times. Among the lazy-loading approaches, Double-Checked Locking proved to be the most performant for repeated access.
- **Thread Safety:** In this specific experimental run, all implementations yielded safe results (1 unique instance). However, the locked implementations (Synchronized, Double-Checked, Bill Pugh) guarantee this safety architecturally.
- **Inconsistencies & Bottlenecks:** The Synchronized method presents a massive performance bottleneck. The empirical data explicitly shows it is $\sim 3\times$ slower during repeated access due to continuous lock acquisition overhead.